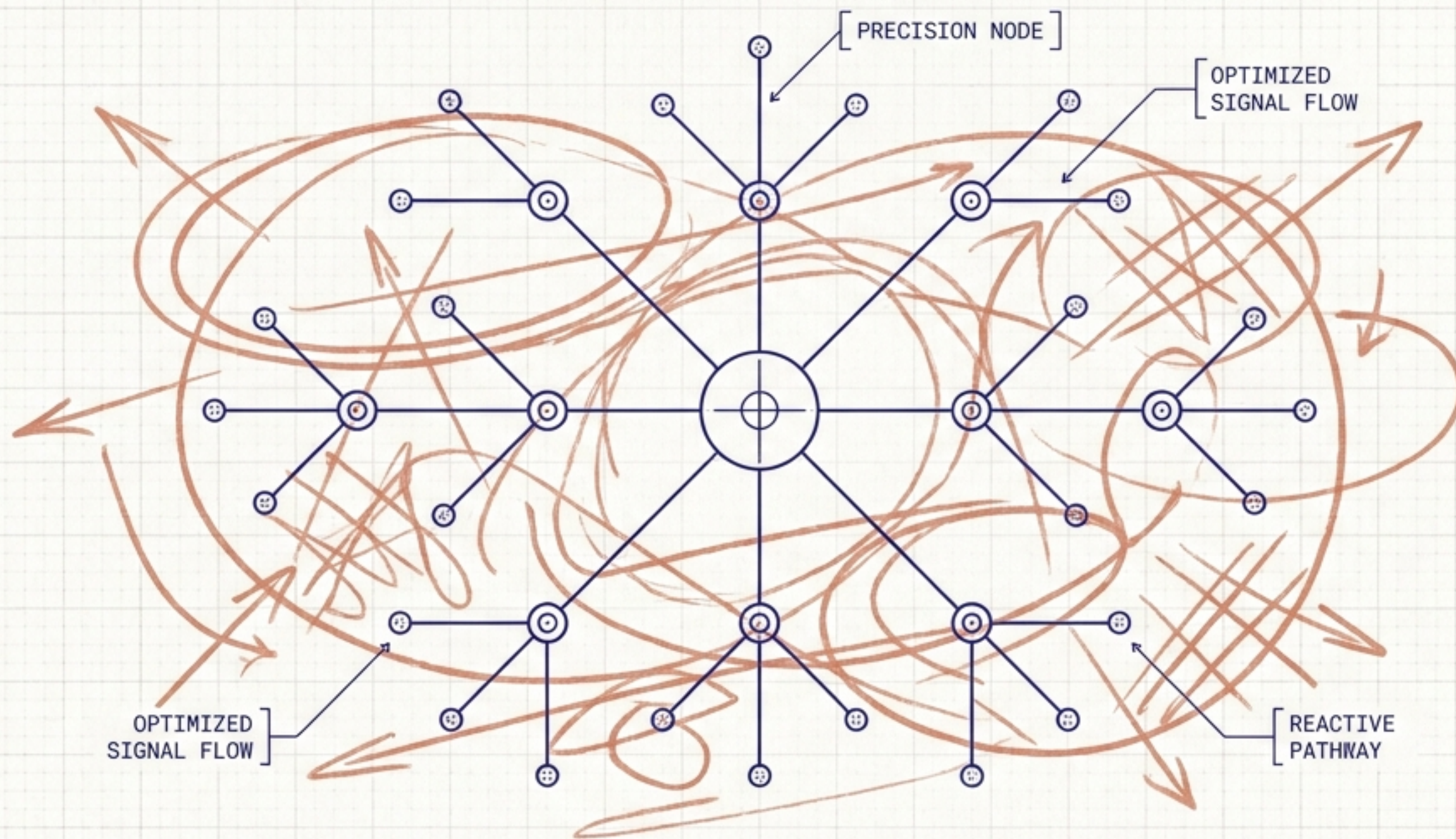


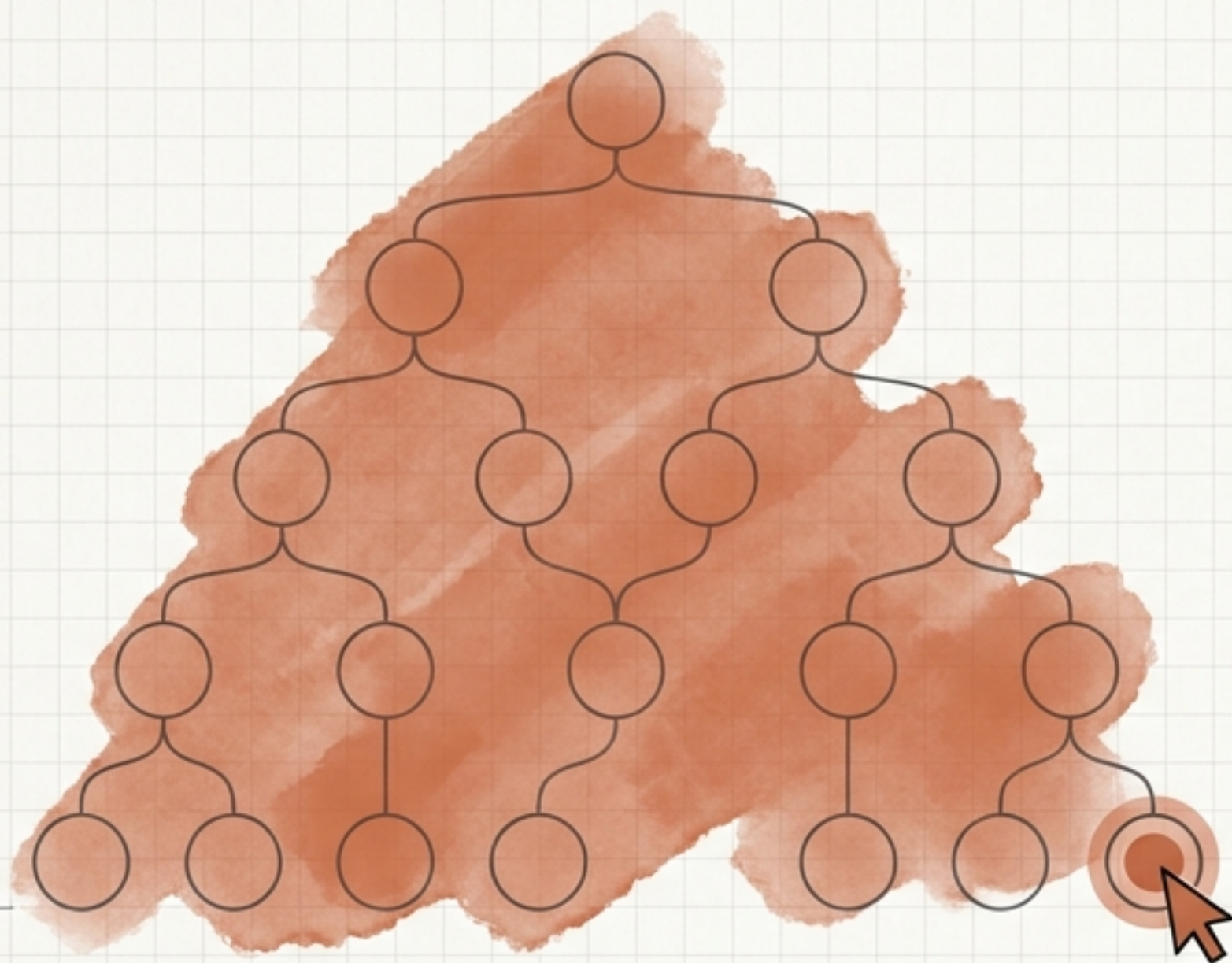
Сигналы в Angular

От глобальных проверок к хирургической точности.

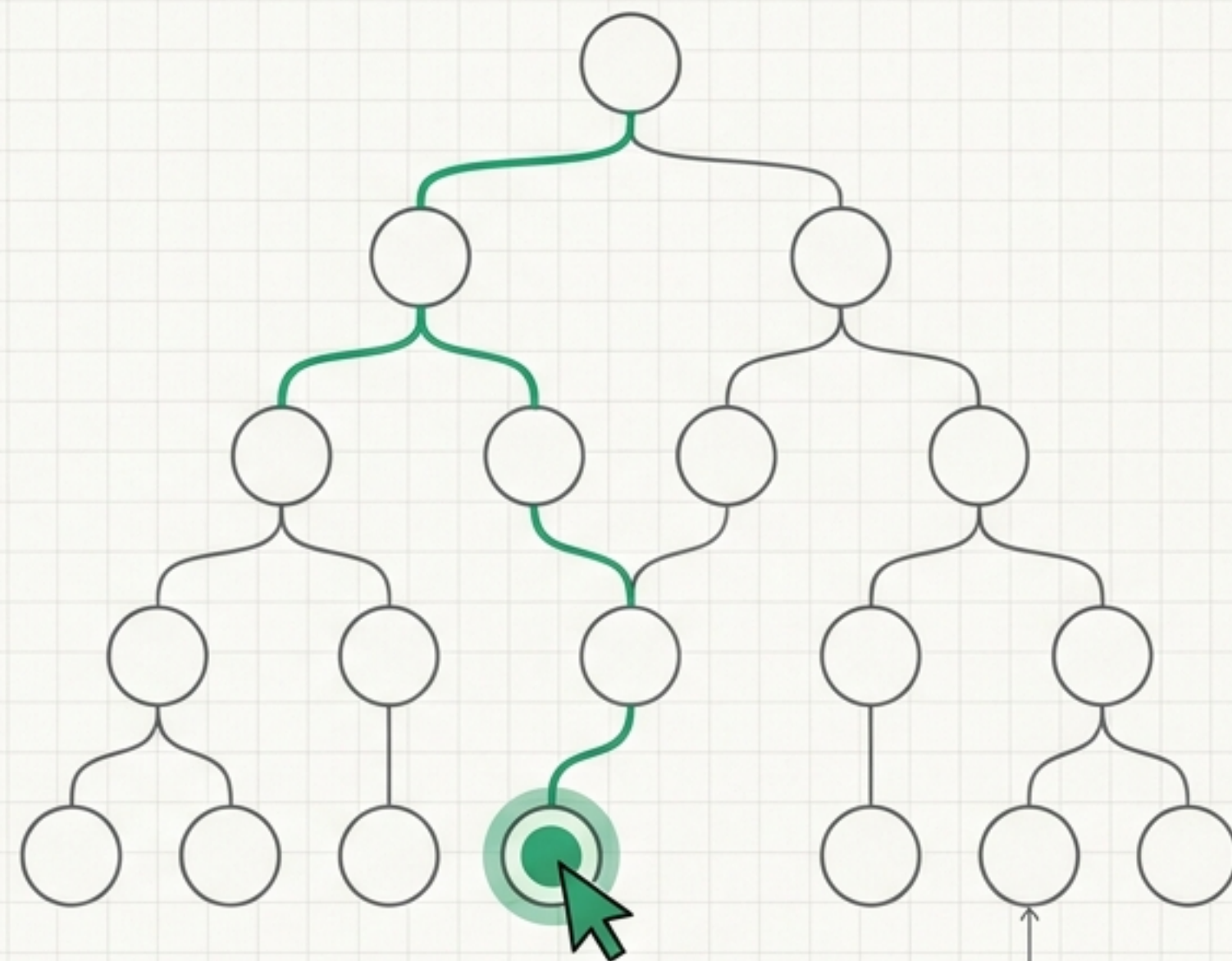


АРХИТЕКТУРНЫЕ ДИАГРАММЫ РЕАКТИВНОСТИ

Zone.js: Слепая проверка всего дерева



Сигналы: Точечная реактивность



Сигнал – это контейнер значения, который знает, кто его читает.
Обновляются только затронутые узлы.

Эволюция индустрии, а не изолированный эксперимент

2010
Knockout.js:
Идея observable
и термин
computed

2016-2021
SolidJS:
Термин signal,
отказ от
Virtual DOM

2022
Preact:
Внедрение сигналов
в готовый
фреймворк

2023+
Angular: Адаптация
push-pull алгоритма
и хирургическая
точность

2015
MobX:
Автоматическое
отслеживание
зависимостей

2020
Vue 3:
Нейминг API
(ref, computed,
effect)

2023+
Angular:
Адаптация push-
pull алгоритма и
хирургическая
точность

Ментальная модель: электронная таблица

`signal()` =
Ячейка с числом
(изменяемое
состояние)

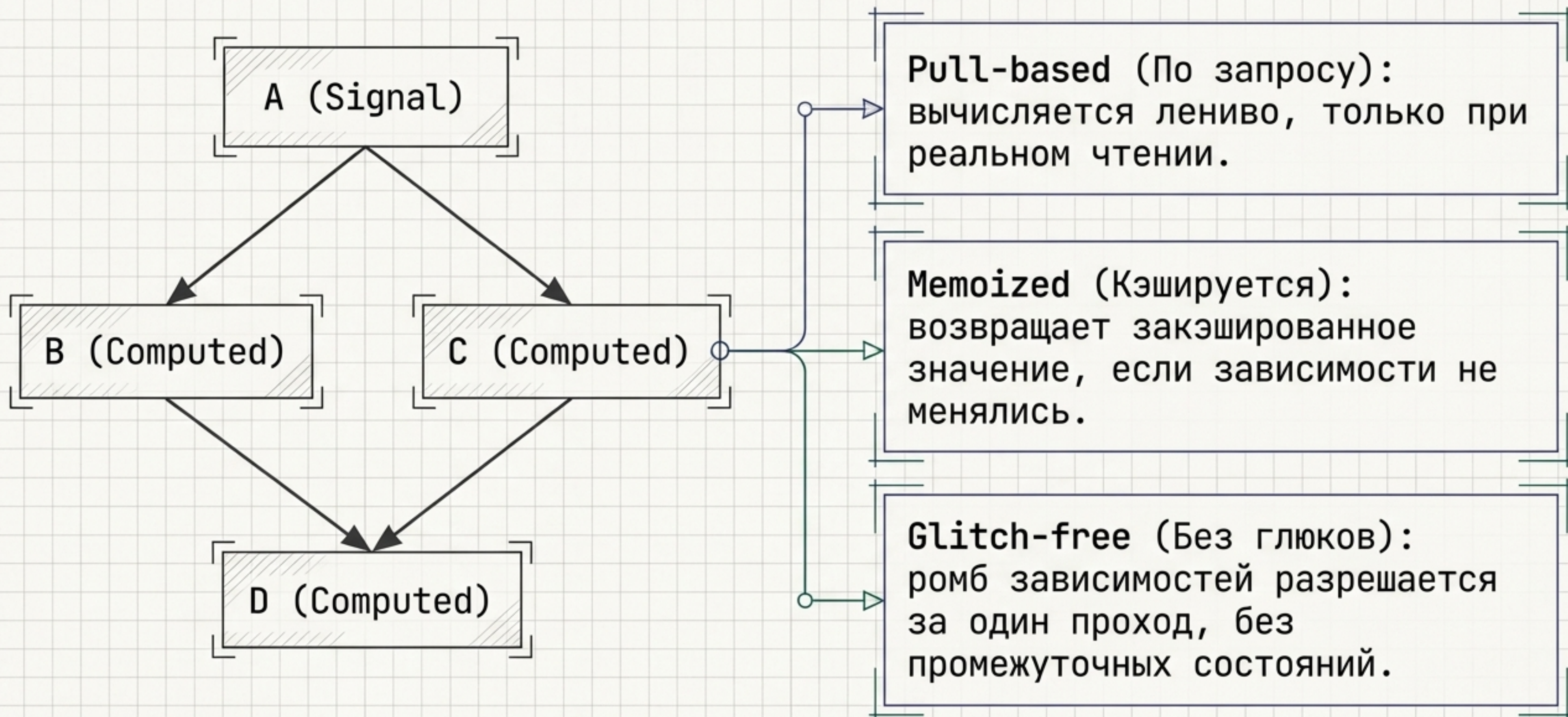
	A	B	C
1	10	20	=A1+B1
2			
3			

`computed()` =
Ячейка с формулой
(пересчитывается сама
при изменении входов)



`effect()` =
Макрос
(побочный эффект при
изменении ячейки)

Граф зависимостей: Алгоритм под капотом



Базовые примитивы: Раньше и Сейчас

Раньше (ООП / Zone.js)

Состояние

```
count = 0;
```

Производные
данные

```
get double() {  
  return this.count * 2;  
}
```

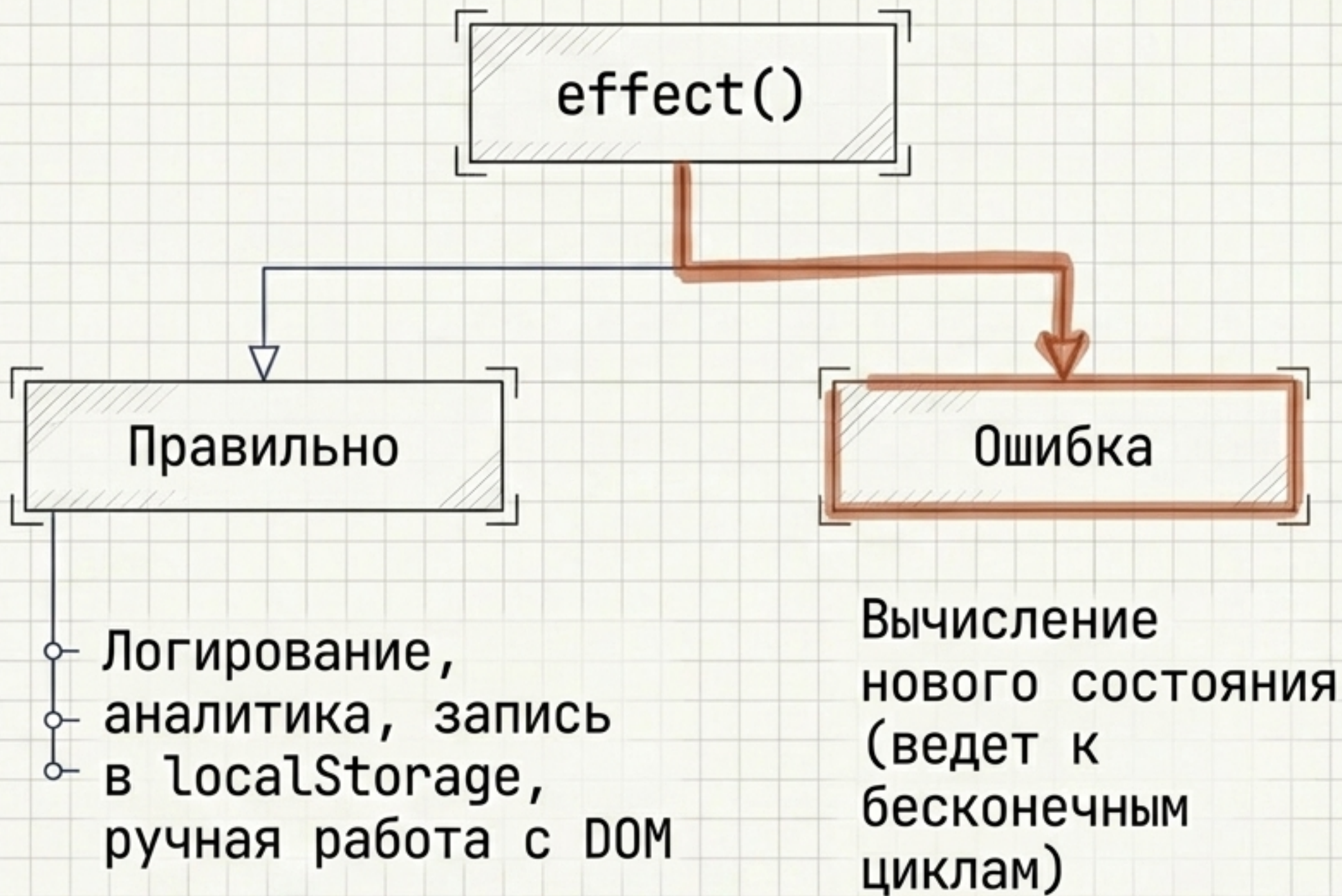
Сейчас (Сигналы)

```
count = signal(0);
```

```
double = computed(() =>  
  this.count() * 2);
```

Инкапсуляция: Менять состояние можно только внутри владельца.
Снаружи – только чтение.

Побочные эффекты и чистота функций

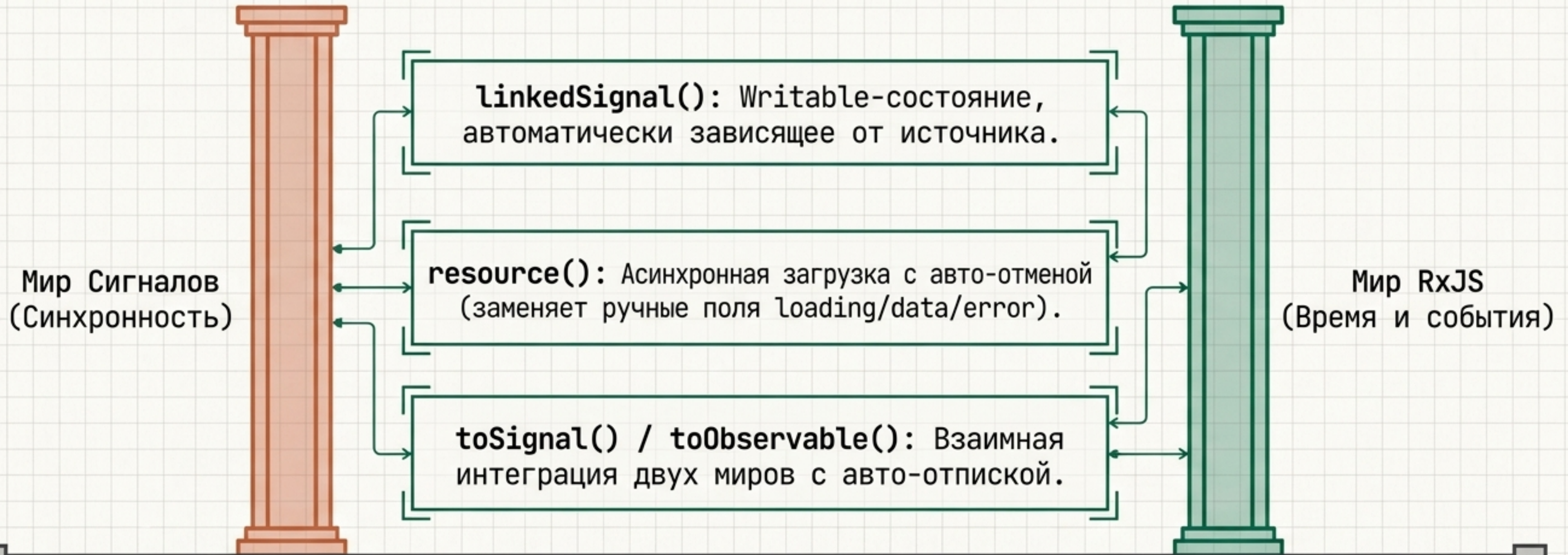


`untracked()`: Чтение значения без создания подписки.

`onCleanup()`: Очистка ресурсов перед каждым перезапуском эффекта.

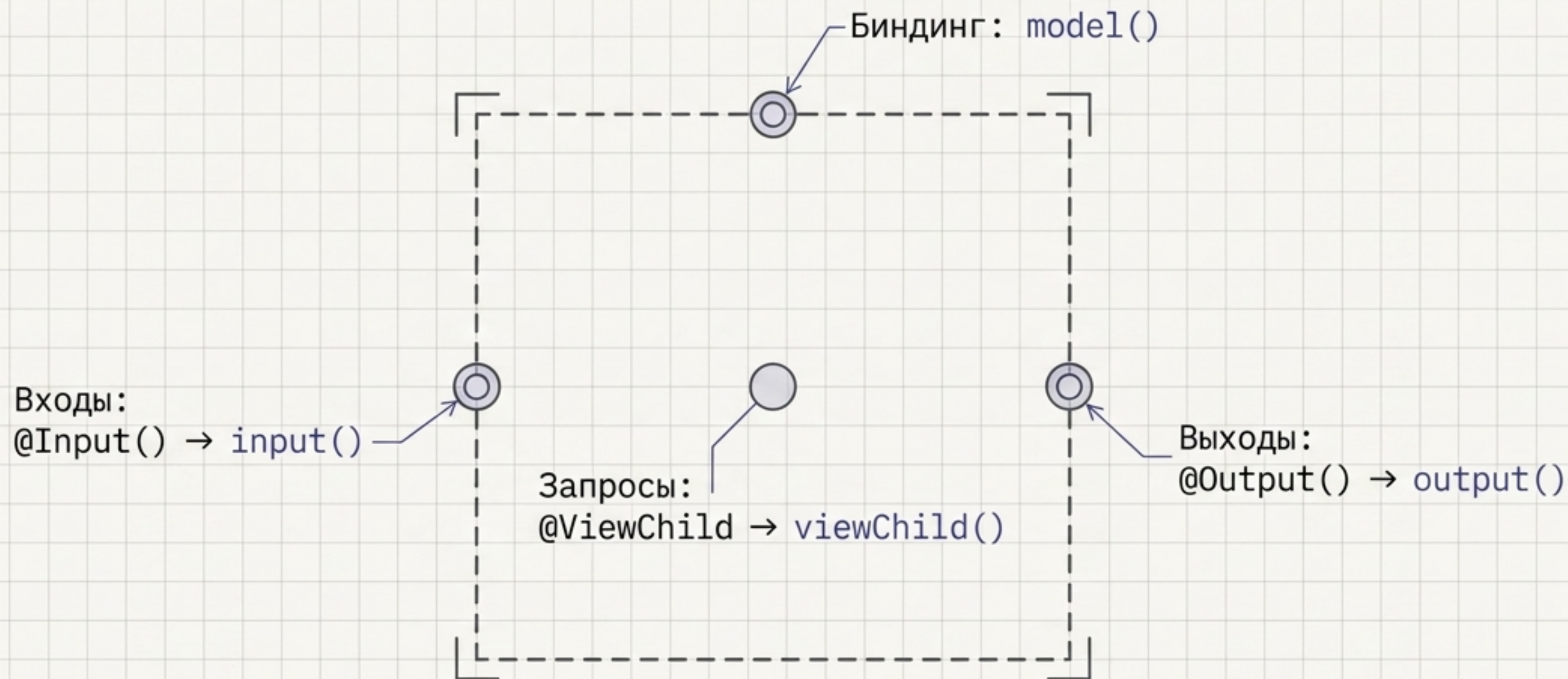
`equal`: Своя проверка равенства для предотвращения лишних обновлений.

Продвинутые примитивы: Мост между мирами



**Золотое правило: Состояние – в сигналах.
Время, события и race conditions – в RxJS.**

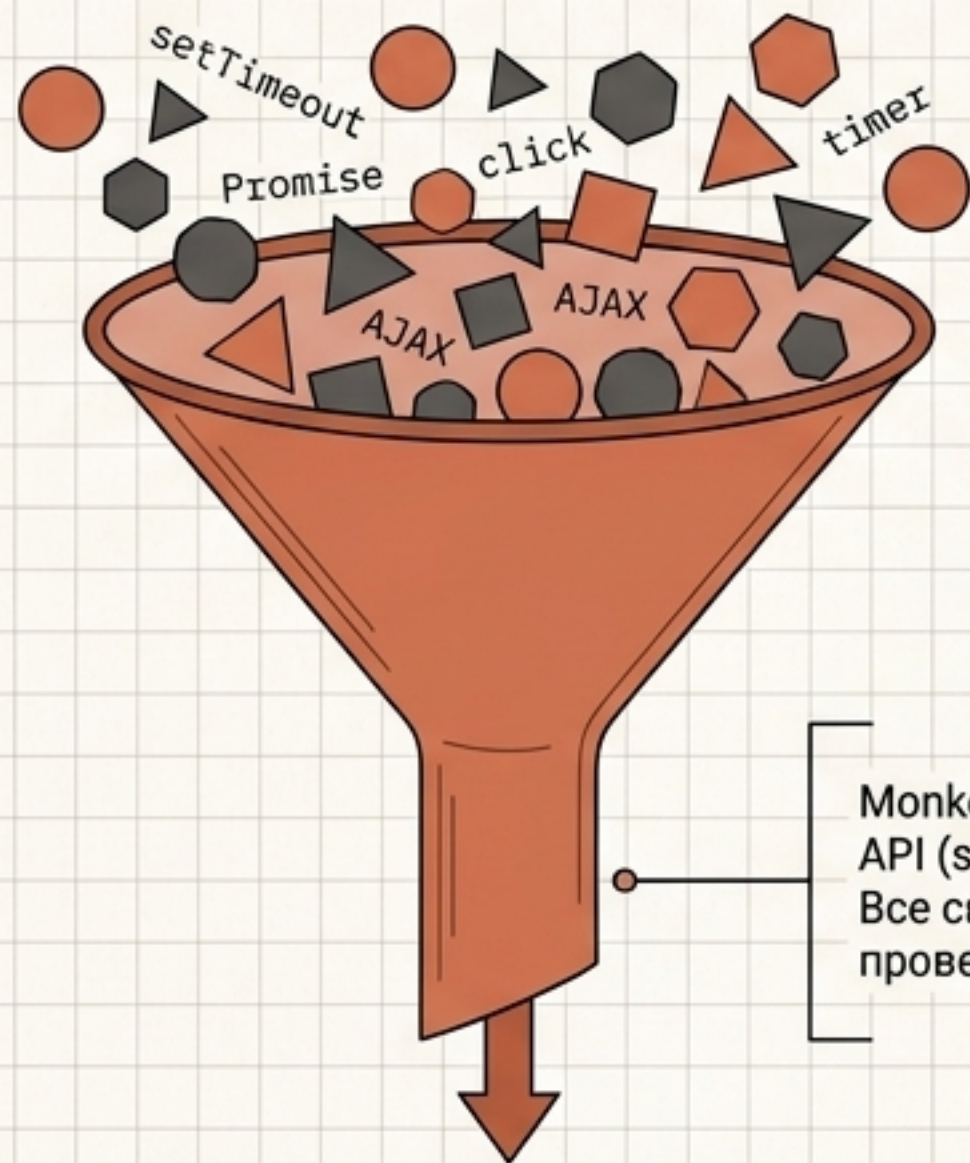
Взаимодействие компонентов: От декораторов к графу



Компоненты больше не ждут хуков жизненного цикла.
Их входы и ссылки на DOM — узлы единого реактивного графа.

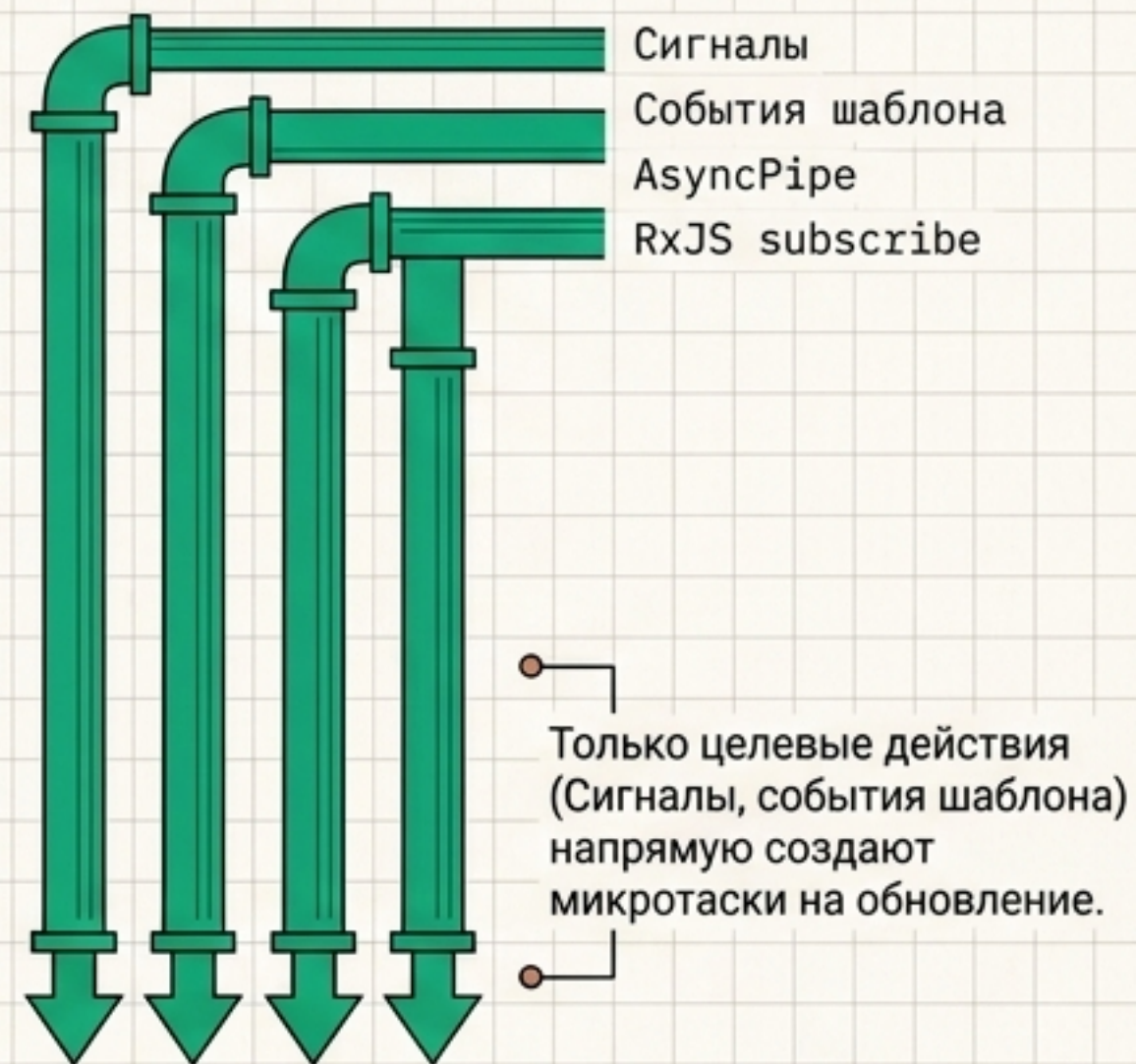
Zone.js против Zoneless

Zone.js: Перехват всего



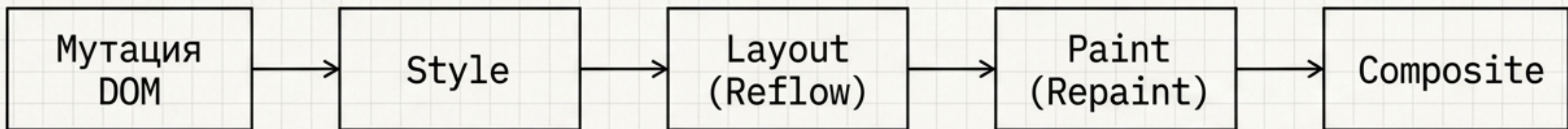
Monkey-patching глобальных API (`setTimeout`, `Promise`, `click`). Все сваливается в глобальную проверку `ApplicationRef.tick()`

Zoneless: Прямое планирование



В режиме `zoneless` изменение обычного поля через `setTimeout` не обновит экран.
Состояние обязано жить в сигналах.

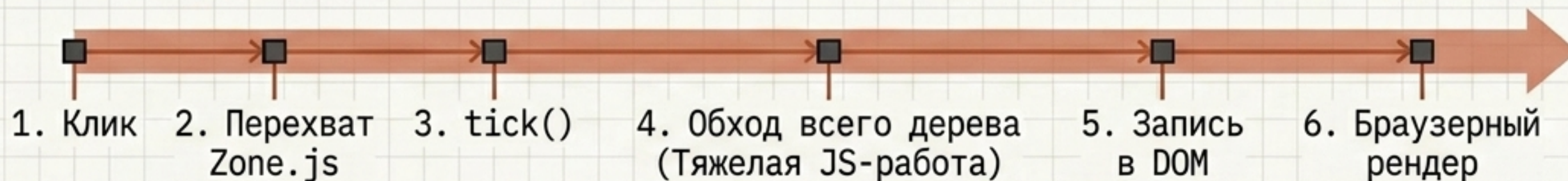
Физическая реальность: Конвейер браузера



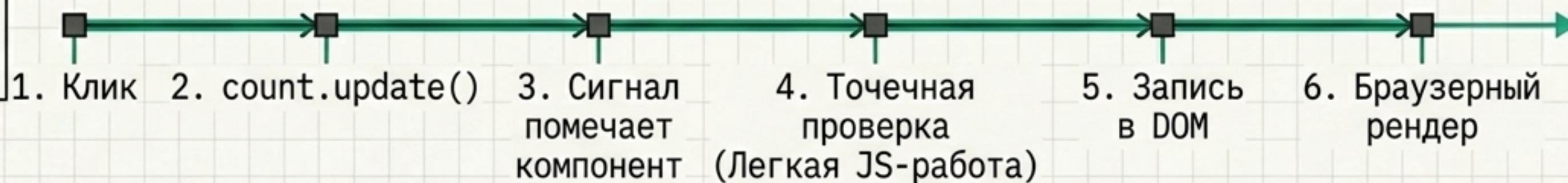
Что изменилось в DOM	Затронутые этапы	Стоимость
Геометрия (width, height, вставка узлов)	Style → Layout → Paint → Composite	Дорого (Reflow)
Внешний вид (color, background)	Style → Paint → Composite	Средне (Repaint)
GPU (transform, opacity)	Только Composite Composite	Дешево

Гонка: От клика до пикселя

Старый путь
(Zone.js)



Новый путь
(Signals + Zoneless)



Этапы работы с DOM и браузером идентичны.
Сигналы экономят время именно на этапе вычислений JavaScript.

Разделение труда: Кто что делает?

JavaScript / Angular:
Обнаружение изменений
(Change Detection)

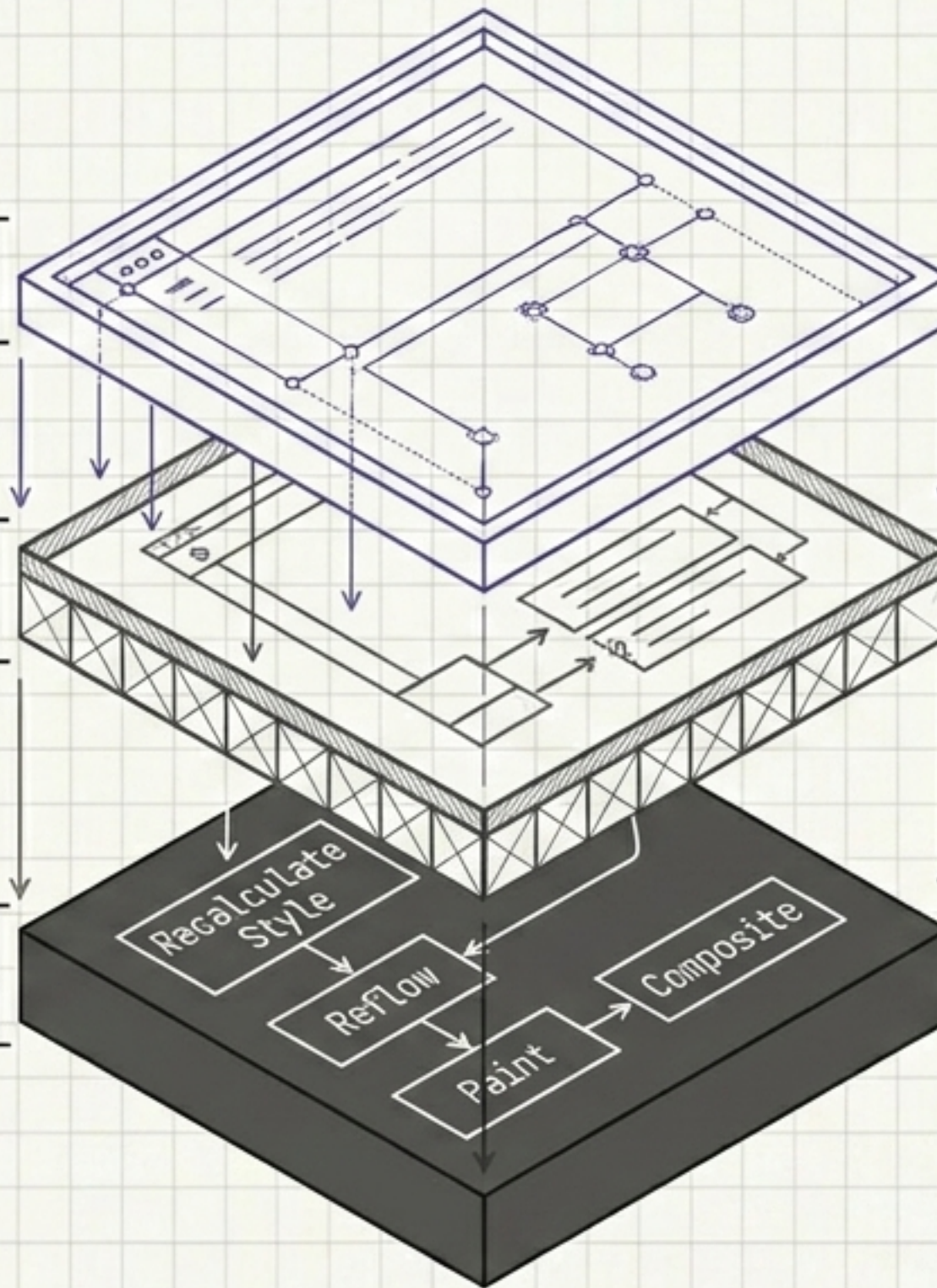
Вычисление биндингов, чтение
сигналов, сравнение значений.
Здесь работают **Сигналы**.

DOM API:
Incremental DOM

Точечная физическая запись
атрибутов, текста, классов.

Браузер / C++:
Pixel Pipeline

Recalculate Style, Reflow
Paint, Composite



Сигналы не ускоряют
браузер. Они
гарантируют, что Angular
не выполнит ни одной
лишней строки JS-кода
перед тем, как передать
работу браузеру.

Архитектура реактивности: Раньше и Сейчас

Аспект	Раньше (Zone.js / ООП)	Сейчас (Zoneless / Сигналы)
Change Detection	Обход всего дерева	Точечное обновление зависящих узлов
Состояние	Обычные поля класса	<code>`signal()`</code>
Производные данные	Геттеры (пересчет каждый CD)	<code>`computed()`</code> (ленивые, кэшируются)
Реакция	<code>ngOnChanges</code> / <code>RxJS subscribe</code>	<code>`effect()`</code>
Входы/Выходы	<code>@Input</code> , <code>@Output</code> , <code>EventEmitter</code>	<code>`input()`</code> , <code>`output()`</code> , <code>`model()`</code>
Запросы к DOM	<code>@ViewChild</code> + <code>ngAfterViewInit</code>	<code>`viewChild()`</code> (реактивный сигнал)
Асинхронность	<code>BehaviorSubject</code> + <code>async pipe</code>	<code>`resource()`</code> / <code>`toSignal()`</code>

Будущее заложено в самом языке



Сигналы выходят за рамки фреймворков. Вскоре они станут нативным примитивом самого языка JavaScript, подобно Promises.

Angular первым из энтерпрайз-фреймворков интегрирует будущий стандарт веба, обеспечивая максимальную производительность без сторонних библиотек.